

Project Proposal, Feature Suite & ERD

eCommerce — Winter 2026

Deliverable	1 of 3
Team Size	3–4 students (self-selected, same Lab section)
Submission	Single PDF document

1. Overview

This is the first of three deliverables that together form the eCommerce Final Project (or LIA). Working in teams of three or four, you will plan, design, and build a fully functional PHP web application for an online service. Remember that eCommerce is a broad concept, therefore I encourage your proposal to go beyond the typical online store.

This deliverable focuses entirely on planning: defining what you will build, identifying its core features, and modeling the underlying data.

2. Team Formation

Each team must have three or four members and be composed of students from the same Lab section. Teams are self-selected. Once submitted, team composition cannot be changed without written approval from the instructor.

- Include each member's full name, section and student ID on the cover page.
- Designate one member as team lead for this deliverable — the instructor's point of contact.
- Only the team leader needs to submit on behalf of the whole team.
- A new team lead will be required for each subsequent deliverable.

3. Technical Requirements

Your final work must be built using the technology stack listed below. More details about technical requirements will be provided by the teacher for the next deliverables. The table below is just for you to be aware of — and make sure you are comfortable with — the core structure of your project.

Component	Requirement
Language	PHP 8+
Framework	PHP Slim Framework (not mandatory, but recommended)
Database	MariaDB (MySQL-compatible)
Architecture	MVC or similar modular architecture
Dependencies	Managed via Composer (PSR-4 autoloading)
Dependency Inj.	DI Container — at minimum for the database object

Component	Requirement
OOP	Classes, inheritance and other OOP features expected
Standards	PSR coding standards (naming, namespaces, DI, documentation)
Templates	Twig (or equivalent template engine)
CSS Framework	Bootstrap, Tailwind CSS or another framework of your choice

Note: Make sure the project you propose is complex enough to meaningfully apply all of the patterns above — a simple CRUD app with one or two entities will not be sufficient.

3.1 Application Structure

Your application must be structured around two distinct interfaces:

Administrative Interface

A back-end area where administrators can log in, manage application content, and handle operations. It must include:

- Secure authentication and authorization with role-based access control.
- Two-factor authentication (2FA) using time-based one-time passwords (TOTP).
- CRUD operations for managing application content.
- Secure file upload functionality (ex.: pictures of your products or services).

User-Facing Interface

A front-end area where users can browse content, manage their session-based transactions, and complete their requests. It must include:

- Content browsing organized by category with AJAX-based live search.
- User registration and login with two-factor authentication (2FA).
- Session-based transaction management.
- Transaction completion workflow with confirmation.

Other Concerns

The following concerns apply to the application as a whole:

- Internationalization (i18n) supporting at least two languages.
- Flash messages for user feedback (success, error, warning, info).
- Service layer for centralized input validation.
- Security best practices: prepared statements, password hashing, and XSS prevention.

4. Required Deliverables

Your PDF submission must contain all of the following sections, in the order listed:

Section	Content Required	Notes
Cover Page	Team member names, student IDs, and section	Just one submission per team
Project Title	A concise, descriptive title for the web application	Also list a few domain names you might want to use for your project.

Section	Content Required	Notes
Project Description	2–3 paragraphs: purpose, target users, and scope	No AI-generated text, please.
Repository URL	Public GitHub / GitLab repo URL — created for this deliverable	See Section 4.3
Feature Suite	Minimum 5 features with short descriptions	Please don't write generic features, I'm asking you to describe how those features relate to your project in particular
ERD	All entities, attributes, PKs/FKs, and cardinality	Embedded image in the PDF file (your DB structure can change over time, that's normal)
Deployment plan	Where you intend to host your project, and what's the deployment workflow	If you don't know much about this topic, do some research or reach out, I'll be happy to help!

4.1 Feature Suite — Guidance

List at least five (5) distinct features your application will support. For each feature, provide a short name and a one- or two-sentence description of what it does and who benefits from it. Features must be meaningful and relevant to your project.

Example features (do not copy verbatim):

- Product Catalogue — Customers can browse by category, search by keyword, and filter by price.
- Shopping Cart — Authenticated users can add, remove, and update items; the cart persists across sessions.
- Order Checkout — A multi-step flow for address entry, review, and order confirmation.
- Admin Dashboard — Administrators manage listings, view orders, and monitor inventory.
- Review & Rating System — Verified buyers can leave star ratings and written reviews.

4.2 ERD — Guidance

Your Entity-Relationship Diagram must model all entities the application will store. At minimum, expect to model users, products, orders, and order items. The ERD must show:

- All entity names with their attributes (including primary keys).
- All foreign-key relationships between entities.
- Cardinality notation (e.g., one-to-many, many-to-many).

Note: You may embed the ERD into your PDF. You can draw it using any tool (draw.io, Lucidchart, dbdiagram.io, etc.).

4.3 GitHub / GitLab Repository — Requirements

Your team must create a public Git repository on GitHub or GitLab and submit its URL as part of this deliverable. The repository must be set up professionally from the start, as the instructor will review it throughout all three deliverables.

Repository Setup

- The repository must be public and accessible without authentication.

- Include a descriptive README.md with the project title, team members, and a brief project description.
- Include an appropriate (open-source preferred) license (e.g., MIT).
- Include a .gitignore file suited for PHP projects.
- The repository structure must be organized and professional — no temp files, or unrelated content.

Workflow & Individual Participation

- Each team member must have their own GitHub / GitLab account and contribute through it.
- Commit messages must be clear and descriptive — avoid vague messages such as “fixed stuff” or “update”.
- Use the platform’s issue tracker to plan and track work items, bugs, and tasks.
- The instructor will review commit history and issue participation to assess each member’s individual contribution. Uneven participation will affect individual grades.

Note: Better to set up the repository as part of Deliverable 1 and keep it active throughout the project. A repository with all commits pushed the night before the final deadline is a red flag.

5. Submission Instructions

- Submit a single PDF file. No Word documents, ZIP archives, or external links to the document itself.
- Include your repository URL clearly on the cover page and in Section 4.3.
- File name format: D1_TeamName_ProjectTitle.pdf (e.g., D1_TeamAlpha_ShopEase.pdf).
- Late submissions are subject to the course’s standard late-penalty policy.

6. Grading Rubric

Criterion	Pts	Description
Project Description Quality	25	Clear purpose, target users, and realistic scope
Feature Suite Completeness	25	5 meaningful features described
ERD Correctness	25	All entities, attributes, PKs/FKs, and cardinality shown
GitHub / GitLab Repository	15	Public, professional setup: README, license, .gitignore, organized structure
Document Formatting	10	Professional PDF, well-structured, complete
TOTAL	100	

7. Academic Integrity

All submitted work must be the original work of the team. You may use diagramming tools, references, and online resources, but may not copy another team’s proposal. Suspected cases of academic dishonesty will be handled in accordance with the Vanier’s academic integrity policy.